



UNAIR
HEBAT

WORLD CLASS UNIVERSITY
#521-530
QS TOP UNIVERSITY 2021

Pertemuan 14

Kompleksitas Algoritma

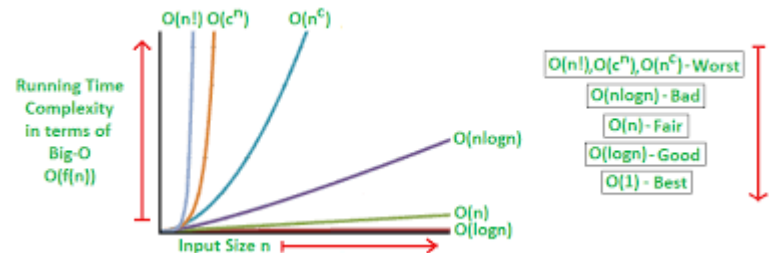
Muhammad Noor Fakhruzzaman, S.Kom., M.Sc.

MK Matematika Diskrit
Prodi S1 Teknologi Sains Data
Fakultas Teknologi Maju dan Multidisiplin
Universitas Airlangga



PENDAHULUAN

- Seperti yang kita tahu, algoritma untuk menyelesaikan suatu permasalahan ada banyak
- “All roads lead to rome”
- Bagaimana cara kita memilih algoritma yang paling baik?
- Apa ukuran algoritma yang baik?



Algoritma yang Baik

1. **Benar**, di mana algoritma menyelesaikan masalah dengan tepat, sesuai dengan definisi masukan / keluaran algoritma yang diberikan.
2. **Efisien**, berarti algoritma menyelesaikan masalah tanpa memberatkan bagian lain dari aplikasi.
3. **Mudah diimplementasikan**, artinya sebuah algoritma yang baik harus dapat dimengerti dengan mudah sehingga implementasi algoritma dapat dilakukan siapapun dengan pendidikan yang tepat, dalam waktu yang masuk akal.

Algoritma yang Baik

- Jarang tercapai ketiga syarat tadi
- Sehingga bermunculan banyak approach / algoritma dalam penyelesaian masalah
- Terdapat permasalahan **Optimasi** algoritma yang berkaitan dengan efisiensi
- Perkembangan zaman diikuti perkembangan device komputer dan algoritma yang semakin efisien

Bagaimana cara mengukur Efisiensi algoritma?

- Contoh ada fungsi sbb $f(x, y) = x^y$
- Untuk menulis fungsi perpangkatan dalam Python (tanpa operator ** dan library lainnya)
- Loop berupa perkalian angka pertama dengan dirinya sendiri sebanyak y

```
def pangkat(x, y):  
    hasil = 1  
    for i in range(0, y):  
        hasil = x * hasil  
    return hasil
```

Pengukuran efisiensi

- Dari fungsi tadi, berapa Langkah yang diperlukan untuk mencapai hasil akhir?
- Penghitungan jumlah Langkah yang diperlukan untuk menyelesaikan satu permasalahan adalah cara menghitung efisiensi paling sederhana

pangkat(2, 1)



```
hasil = 1
for i in range(0, 1):
    hasil = 2 * hasil
return hasil
```



```
hasil = 1
hasil = 2 * hasil
return hasil
```

Pengukuran efisiensi

- Bagaimana jika pangkat(2,2)?

```
hasil = 1
for i in range(0, 2):
    hasil = 2 * hasil
return hasil
```



```
hasil = 1
hasil = 2 * hasil
hasil = 2 * hasil
return hasil
```

- Bagaimana jika pangkat (2,5)?

```
hasil = 1
hasil = 2 * hasil
hasil = 2 * hasil
hasil = 2 * hasil
hasil = 2 * hasil
hasil = 2 * hasil
return hasil
```

Pengukuran efisiensi

- Dapat dilihat bahwa dalam fungsi pangkat, loop $\text{hasil} = x * \text{hasil}$ akan dijalankan sebanyak y kali
- Berapa Langkah fungsi tersebut berjalan untuk memangkatkan angka x dengan y ?

Baris Kode	Jumlah Eksekusi / Langkah
<code>hasil = 1</code>	1
<code>hasil = x * hasil</code>	y
<code>return hasil</code>	1

Pengukuran efisiensi

- Fungsi pangkat tadi akan selalu diselesaikan dalam $2 + y$ Langkah

Y	Proses Perhitungan	Jumlah Langkah
1	$2 + 1$	3
10	$2 + 10$	12
100	$2 + 100$	102
1000	$2 + 1000$	1002
10000	$2 + 10000$	10002

- Semakin besar nilai y , nilai 2 menjadi tidak relevan. Sehingga cukup dinotasikan sebagai y Langkah saja
- Semakin besar y , semakin banyak Langkah yang diperlukan

- Fungsi pangkat tadi memiliki grafik pertumbuhan kompleksitas secara Linear



Notasi Asimtotik

- Fungsi pertumbuhan tadi penting untuk melihat efisiensi algoritma yang relative terhadap jumlah data yang akan diproses (dalam kasus tadi adalah y)
- Diperlukan notasi khusus, yaitu notasi asimtotik
- Yang akan digunakan dalam pengukuran efisiensi algoritma adalah notasi Big-O (The Big-O Notation)
- Notasi Big-O digunakan untuk mengkategorikan algoritma ke dalam fungsi yang menggambarkan batas atas (*upper limit*) dari pertumbuhan sebuah fungsi ketika masukan dari fungsi tersebut bertambah banyak.

The Big-O Notation

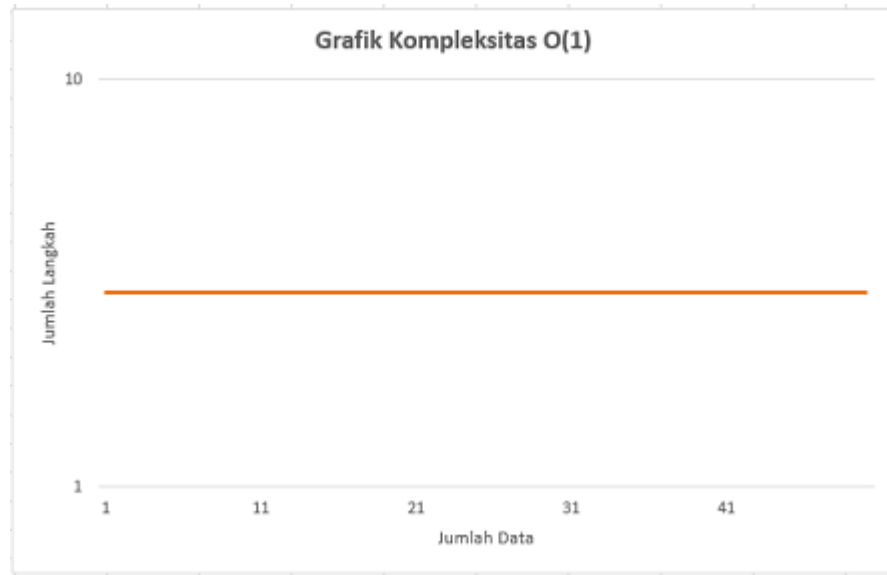
- Perhitungan jumlah langkah dan pertumbuhannya yang kita lakukan pada bagian sebelumnya merupakan langkah-langkah untuk mendapatkan fungsi Big-O dari sebuah algoritma.
- Big-O, seperti namanya, dituliskan sebagai fungsi “O” dengan nilai masukan berupa tingkat pertumbuhan dari fungsi yang dijabarkan. Misalnya, algoritma perpangkatan dengan pertumbuhan linear yang kita kembangkan pada bagian sebelumnya memiliki kelas Big-O $O(n)$

Kelas dalam Big-O yang sering ditemui

Fungsi Big-O	Nama
$O(1)$	Konstan
$O(\log n)$	Logaritmik
$O(n)$	Linear
$O(n \log n)$	$n \log n$
$O(n^2)$	Kuadratik
$O(n^m)$	Polinomiale
$O(n!)$	Faktorial

$O(1)$ Konstan

- Algoritma paling Efisien karena tidak terpengaruh pada jumlah data / perulangan



$O(\log n)$: Kompleksitas Logaritmik

- Algoritma yang menyelesaikan masalah dengan membagi-bagi masalah tersebut menjadi beberapa bagian, sehingga masalah dapat diselesaikan tanpa harus melakukan komputasi atau pengecekan terhadap seluruh masukan

```
def binary_search(lst, search):  
    lower_bound = 0  
    upper_bound = len(lst) - 1  
  
    while True:  
        if upper_bound < lower_bound:  
            print("Not found.")  
            return -1  
  
        i = (lower_bound + upper_bound) // 2  
  
        if lst[i] < search:  
            lower_bound = i + 1  
        elif lst[i] > search:  
            upper_bound = i - 1  
        else:  
            print("Element " + str(search) + " in " + str(i))  
            return 0
```

Breakdown

1. Langkah yang akan selalu dieksekusi pada awal fungsi, yaitu inialisasi `lower_bound` dan `upper_bound`: **2 langkah.**
2. Pengecekan kondisi `while` (pengecekan tetap dilakukan, walaupun tidak ada perbandingan yang dijalankan): **1 langkah.**
3. Pengecekan awal (`if upper_bound < lower_bound`): **1 langkah.**
4. Inialisasi `i`: **1 langkah.**
5. Pengecekan kondisi kedua (`if lst[i] < search: ...`), kasus terburuk (masuk pada `else` dan menjalankan kode di dalamnya): **4 langkah.**

- jika elemen belum ditemukan maka kita akan kembali ke langkah kedua
- kita minimal harus menjalankan **2 langkah** dan pada setiap perulangan while kita menjalankan **7 Langkah**

$$f(n) = 2 + 7(\text{jumlah perulangan})$$

- Berapa kali perulangan?
- Perulangan ditentukan oleh 2 kondisi:
 - Kondisi `if upper_bound < lower_bound:`
 - Kondisi dimana search query ditemukan
 - Kondisi ketika i adalah index tengah `i = (lower_bound + upper_bound) // 2`

- Karena setiap binary search jumlah data dibagi 2 (dari persamaan i tadi) maka

$$f(n) = 7f\left(\frac{n}{2}\right)$$

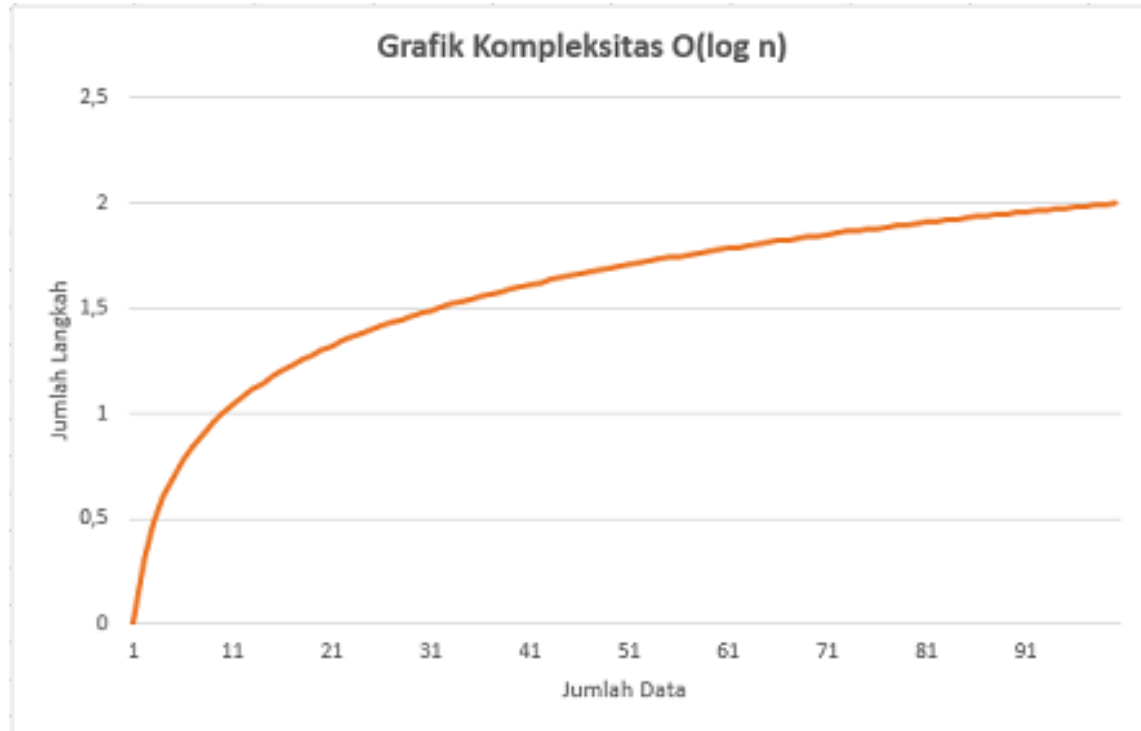
- Konstanta 2 diawal dihilangkan karena tidak relevan jika data besar

$$f(n) = 7f\left(\frac{n}{2}\right) \longrightarrow \begin{aligned} f(n) &= 7f\left(\frac{n}{2}\right) \\ &= 7 * (7f\left(\frac{n}{4}\right)) \\ &= 49f\left(\frac{n}{4}\right) \\ &= 49 * (7f\left(\frac{n}{8}\right)) \\ &\dots \\ &= 7^k f\left(\frac{n}{2^k}\right) \end{aligned}$$

- Karena perulangan akan berhenti jika sisa elemen dalam list = 1, maka
- Jadi, $O(\log_2 n)$ atau $O(\log n)$

$$\begin{aligned} \frac{n}{2^k} &= 1 \\ n &= 2^k \\ \log_2 n &= k \end{aligned}$$

Grafik $O(\log n)$



$O(n)$ Kompleksitas Linear

- Algoritma dengan kompleksitas linear bertumbuh selaras dengan pertumbuhan ukuran data.
- Jika algoritma ini memerlukan 10 langkah untuk menyelesaikan kalkulasi data berukuran 10, maka ia akan memerlukan 100 langkah untuk data berukuran 100.
- Seperti penghitungan efisiensi fungsi pangkat tadi
- Contoh lain, Linear Search

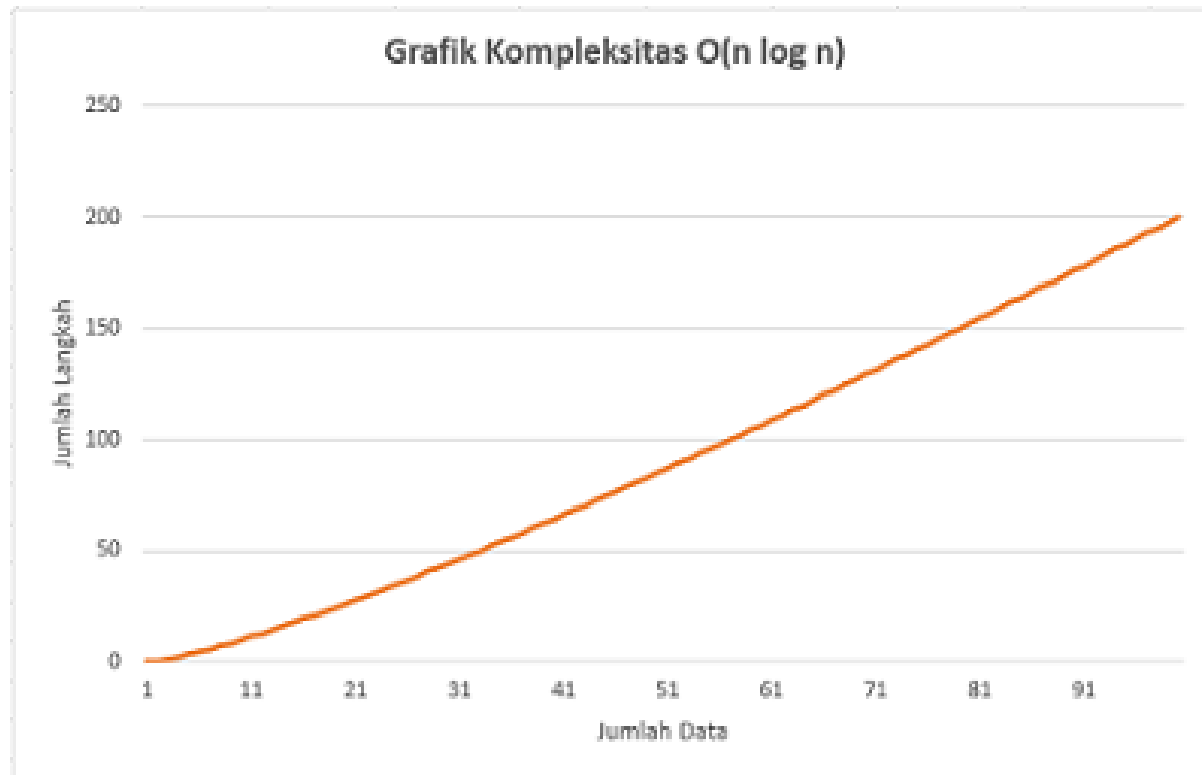
$O(n \log n)$

- Cara penghitungan sama dengan kompleksitas logaritmik yang dilakukan sebanyak n kali
- Misalkan sebuah fungsi untuk mencari sepasang bilangan di dalam sebuah list yang jika ditambahkan akan bernilai 0 (list sudah sorted)

```
def zero_sum(lst):  
    n = len(lst)  
    for i in range(0, n):  
        j = binary_search(lst, -1 * lst[i])  
        if j > i:  
            n1 = str(lst[i])  
            n2 = str(lst[j])  
            print("Zero sum: " + n1 + " and " + n2 + "\n")
```

$O(n \log n)$

- Salah satu solusi yang paling sederhana ialah dengan menelusuri seluruh list, satu demi satu (kompleksitas $O(n)$)
- Lalu mencari pasangannya (nilai invers) dengan binary search (kompleksitas $O(\log n)$)
- Maka, binary search akan diulang sebanyak n kali (dari setiap linear search, mencari pasangan inversnya dengan binary search)
- Jadi kompleksitas binary search $\log n$, dilakukan sebanyak n ,
- Maka, $O(n \log n)$



$O(n^m)$ Kompleksitas Polinomial

- tidak efisien, karena memerlukan jumlah langkah penyelesaian yang jauh lebih besar daripada jumlah data.

```
def kali(a, b):  
    res = 0  
    for i in range(a):  
        for j in range(b):  
            res += 1  
    return res
```

Kode

Jumlah Langkah

for i in range(b):

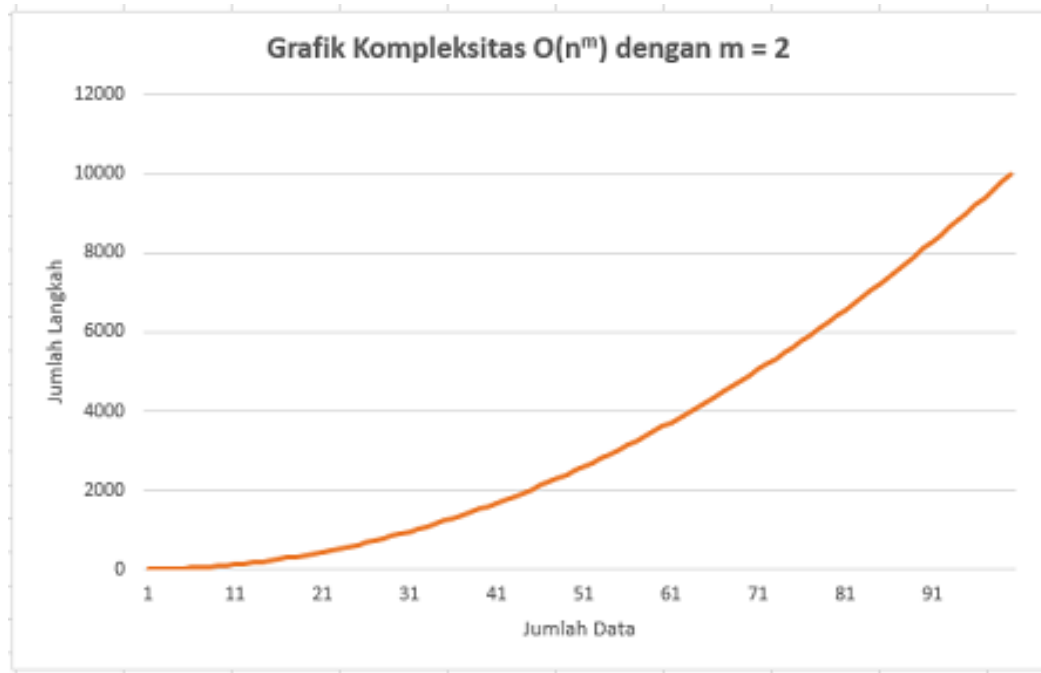
a

res += 1

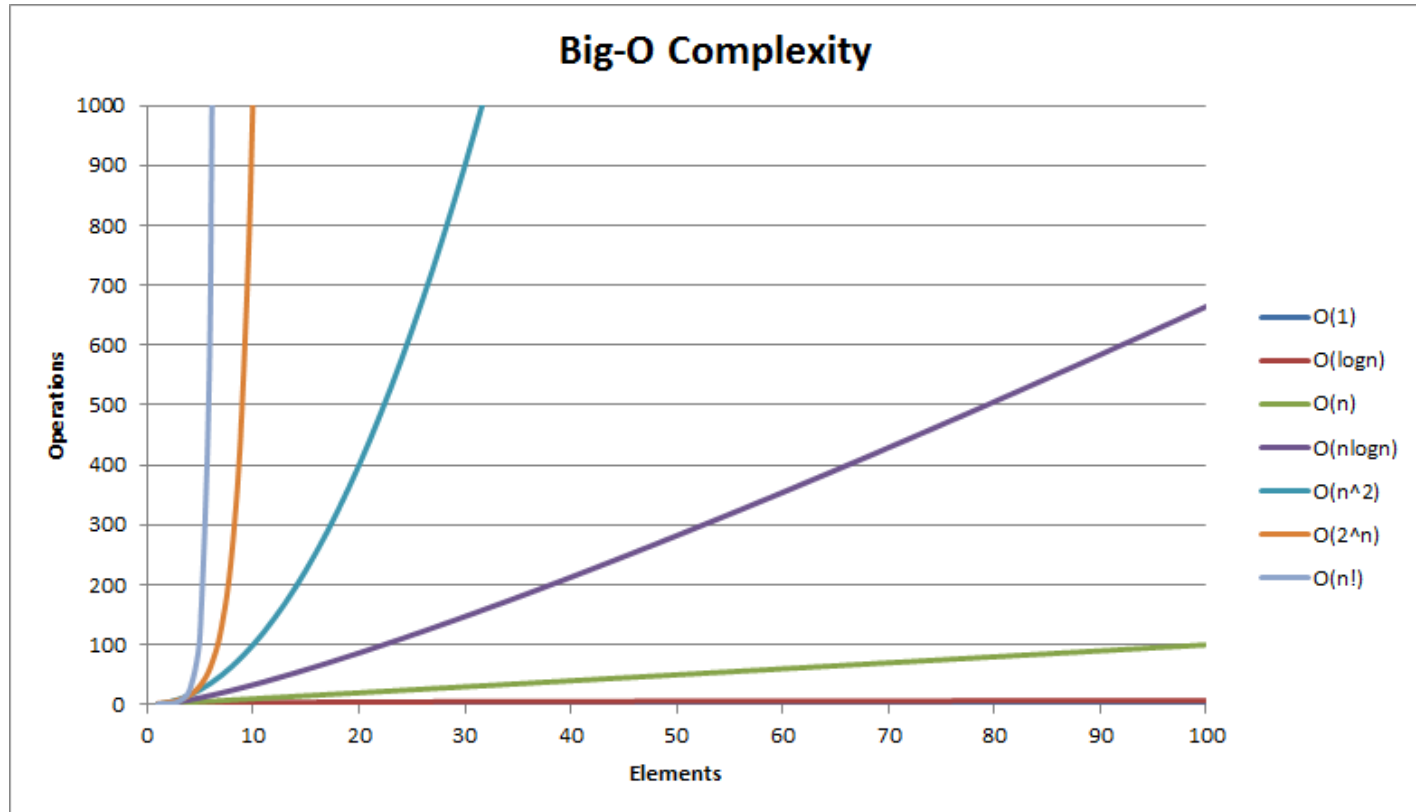
b

- Perkalian a dengan b , dengan cara menambahkan 1 sebanyak b kali yang hasilnya ditambahkan sebanyak a kali

- Jumlah Langkah $a * b$
- Yang jika keduanya sama maka a^2 atau n^2



Perbandingan Kompleksitas



Conclusion

- Pengembangan algoritma idealnya diusahakan mendapatkan kompleksitas $O(1)$ atau $O(\log n)$.
- Jika tidak dapat mencapai kompleksitas maksimal, hal terbaik yang dapat kita lakukan ketika mengembangkan solusi dari masalah adalah melihat apakah masalah yang ada dapat diselesaikan dengan algoritma yang ada terlebih dahulu, sebelum mengembangkan algoritma baru.

Terima Kasih